

习 题

1. 设将整数1, 2, 3, 4依次进栈, 但只要出栈时栈非空, 则可将出栈操作按任何次序夹入其中, 请回答下述问题:

(1) 若入、出栈次序为Push(1), Pop(), Push(2), Push(3), Pop(), Pop(), Push(4), Pop(), 则出栈的数字序列为何(这里Push(i)表示i进栈, Pop()表示出栈)?

(2) 能否得到出栈序列1423和1432? 并说明为什么不能得到或者如何得到。

(3)请分析 1, 2, 3, 4 的24种排列中, 哪些序列是可以通过相应的入栈出栈操作得到的。

2. 循环队列的优点是什么? 如何判别它的空和满?

3. 设长度为n的链队用单循环链表表示, 若设头指针, 则入队出队操作的时间为何? 若只设尾指针呢?

4. 指出下述程序段的功能是什么?

(1) void Demo1(SeqStack *S){

int i; arr[64] ; n=0 ;

while (StackEmpty(S))

arr[n++]=Pop(S);

for (i=0, i< n; i++)

Push(S, arr[i]);

} //Demo1

(2) SeqStack S1, S2, tmp;

DataType x;

//假设栈tmp和S2已做过初始化

while (! StackEmpty (&S1))

{ x=Pop(&S1) ;

Push(&tmp,x); }

while (! StackEmpty (&tmp))

{ x=Pop(&tmp);

Push(&S1,x);

Push(&S2, x); }

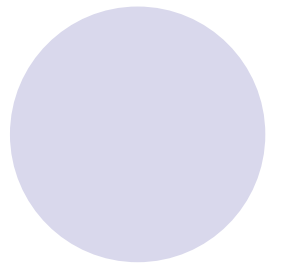
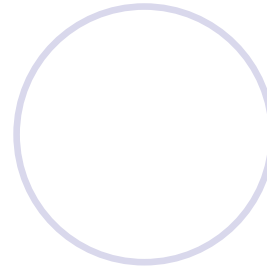
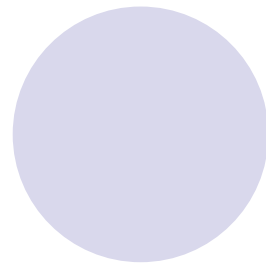
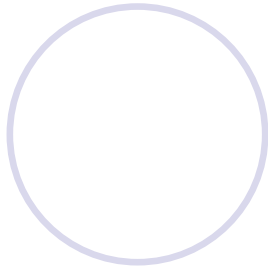
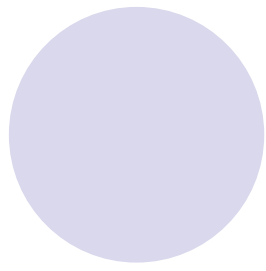
```
(3) void Demo2( SeqStack *S, int m)
{ // 设DataType 为int 型
    SeqStack T; int i;
    InitStack (&T);
    while (! StackEmpty( S))
        if(( i=Pop(S)) !=m) Push( &T,i);
    while (! StackEmpty( &T))
    { i=Pop(&T); Push(S,i);
    }
}
```

Four circles are arranged horizontally at the top of the slide. From left to right: a solid light purple circle, an empty light purple circle, a solid light purple circle, and an empty light purple circle.

```
(4)void Demo3( CirQueue *Q)
{ // 设DataType 为int 型
  int x; SeqStack S;
  InitStack( &S);
  while (! QueueEmpty( Q ))
    {x=DeQueue( Q); Push( &S,x);}
  while (! StackEmpty( &s))
    { x=Pop(&S); EnQueue( Q,x );}
} // Demo3
```



```
(5) CirQueue Q1, Q2;  
    // 设DataType 为int 型  
    int x, i , n= 0;  
    // 设Q1已有内容,  Q2已初始化过  
    while ( ! QueueEmpty( &Q1) )  
        { x=DeQueue( &Q1 ) ;  
          EnQueue(&Q2, x); n++;}  
    for (i=0; i< n; i++)  
        { x=DeQueue(&Q2) ;  
          EnQueue( &Q1, x) ;  
          EnQueue( &Q2, x);}
```



**5、回文是指正读反读均相同的字符序列，
如"abba"和"abdba"均是回文，但"good"不
是回文。试写一个算法判定给定的字符向
量是否为回文。(提示：将一半字符入栈)**

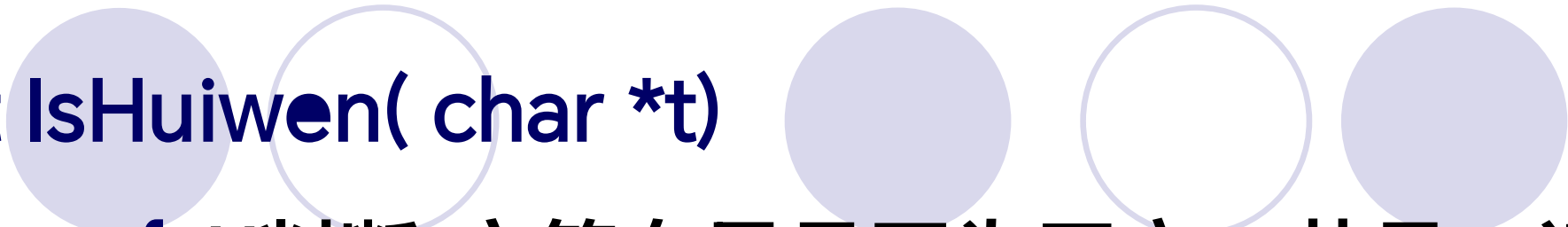


//以下为顺序栈的存储结构定义

```
#define StackSize 100 //假定预分配的栈  
空间最多为100个元素
```

```
typedef char DataType; //假定栈元素的  
数据类型为字符
```

```
typedef struct{  
    DataType data[StackSize];  
    int top;  
}SeqStack;
```



```
int IsHuiwen( char *t)
```

```
{ //判断t字符向量是否为回文，若是，返回1， 否则返回0
```

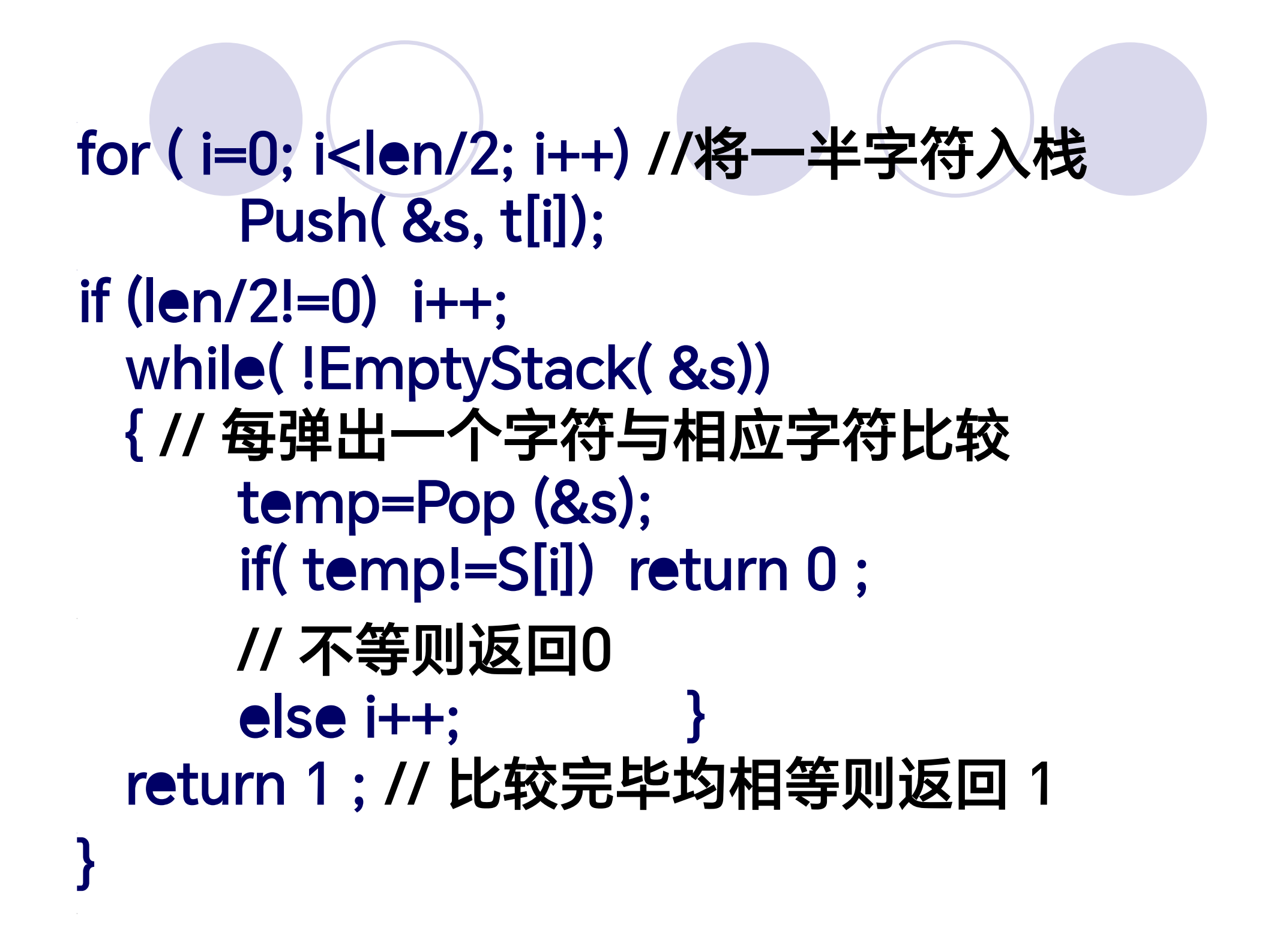
```
SeqStack s;
```

```
int i , len;
```

```
char temp;
```

```
InitStack( &s);
```

```
len=strlen(t); //求向量长度
```



```
for ( i=0; i<len/2; i++) //将一半字符入栈
    Push( &s, t[i]);
if (len/2!=0) i++;
while( !EmptyStack( &s))
{ // 每弹出一个字符与相应字符比较
    temp=Pop (&s);
    if( temp!=S[i]) return 0 ;
    // 不等则返回0
    else i++;
}
return 1 ; // 比较完毕均相等则返回 1
}
```

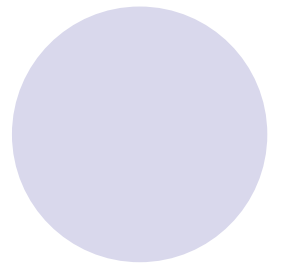
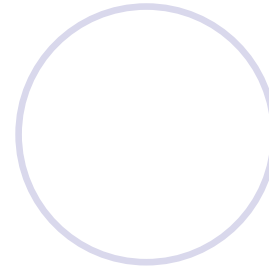
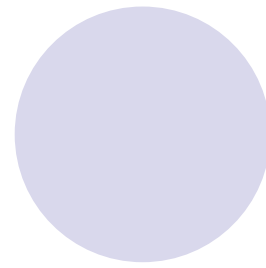
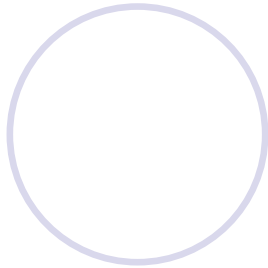
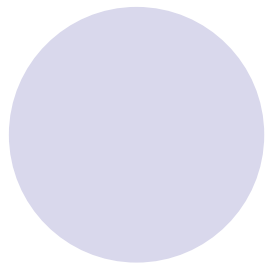
6. 利用栈的基本操作，写一个将栈S中所有结点均删去的算法

void ClearStack(SeqStack *S),
并说明S为何要作为指针参数?

7. 设计算法判断一个算术表达式的圆括号是否正确配对。(提示：对表达式进行扫描，凡遇到‘(’就进栈，遇‘)’就退掉栈顶的‘(’，表达式被扫描完毕，栈应为空。



```
int PairBracket( char *SR)
{ //检查表达式ST中括号是否配对
  SeqStack S; //定义一个栈
  InitStack (&s);
  for (i=0; i<strlen(SR) ; i++){
    if ( S[i]=='(' ) Push(&S, SR[i]);
    //遇'('时进栈
    if ( S[i]==')' ) //遇')'
```



```
if (!StackEmpty(S))
```

```
    Pop(&s); //栈不为空时，将栈顶元素出栈
```

```
else return 0; //不匹配，返回0
```

```
}
```

```
if (EmptyStack(&s)) return 1;
```

```
    // 匹配，返回1
```

```
else return 0; //不匹配，返回0
```

```
}
```

8. 一个双向栈S是在同一向量空间内实现的两个栈，它们的栈底分别设在向量空间的两端。试为此双向栈设计初始化InitStack(S)、入栈Push(S, i, x)和出栈Pop(S, i)等算法，其中i为0或1，用以表示栈号。

9. Ackerman 函数定义如下：请写出递归算法。

$$\text{AKM}(m,n)=\begin{cases} n+1 & \text{当 } m=0 \text{ 时} \\ \text{AKM}(m-1, 1) & \text{当 } m \neq 0, n=0 \text{ 时} \\ \text{AKM}(m-1, \text{AKM}(m, n-1)) & \text{当 } m \neq 0, n \neq 0 \text{ 时} \end{cases}$$

10. 对于循环向量中的循环队列，写出求队列长度的公式。

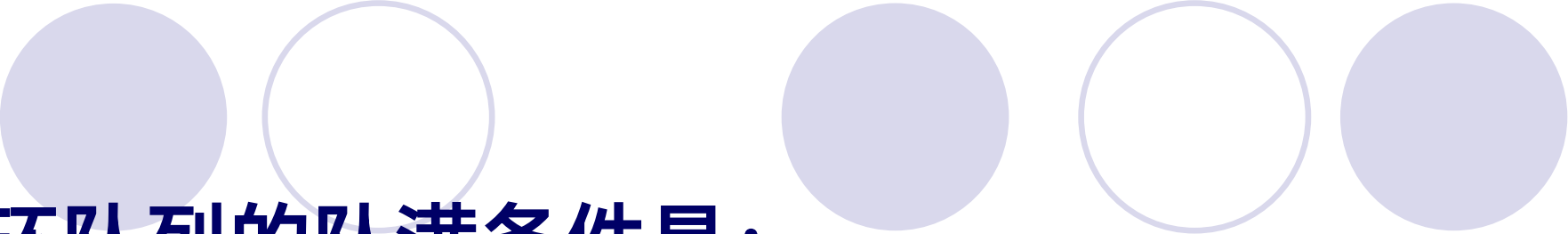
11. 假设循环队列中只设rear和quelen 来分别指示队尾元素的位置和队中元素的个数，试给出判别此循环队列的队满条件，并写出相应的入队和出队算法，要求出队时需返回队头元素。



```
#define QueueSize 100
```

```
typedef char Datatype ; //设元素的类型为  
char型
```

```
typedef struct {  
    int quelen;  
    int rear;  
    Datatype Data[QueueSize];  
}CirQueue;  
CirQueue *Q;
```



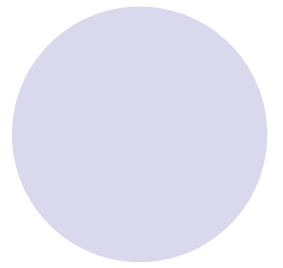
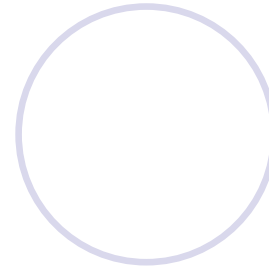
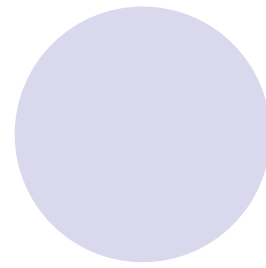
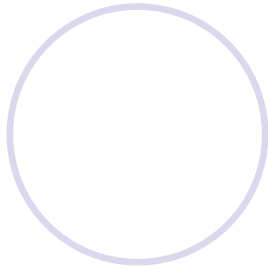
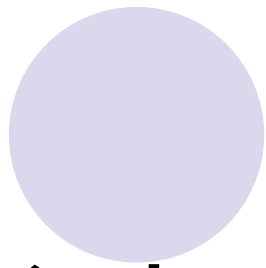
循环队列的队满条件是：

`Q->quelen==QueueSize`

(1)判断队满

`int FullQueue( CirQueue *Q)`

```
{ //判队满,队中元素个数等于空间大小  
  return Q->quelen==QueueSize;  
}
```



(2) 入队

```
void EnQueue( CirQueue *Q, Datatype x)
```

```
{ // 入队
```

```
    if(FullQueue( Q))
```

```
        Error("队已满，无法入队");
```

```
    Q->rear=(Q->rear+1)%QueueSize;
```

```
    Q->Data[Q->rear]=x;
```

```
    Q->quelen++; }
```



(3)出队

Datatype DeQueue(CirQueue *Q)

{ //出队

if(Q->quelen==0)

Error("队已空，无元素可出队");

int tmpfront; //设一个临时队头指针

tmpfront=(QueueSize+Q->rear - Q
->quelen+1)%QueueSize

Q->quelen--;

return Q->Data[tmpfront]; }

第五章 数组

1. 设 $A[0..9, 0..9]$ 是一个 10×10 对称矩阵，采用压缩存储方式存储其下三角部分，已知每个元素占用两个存储单元，其第一个元素 $A_{0,0}$ 的存储位置为1000，求如下问题的计算过程及结果：
- 1) 给出 $A_{4,5}$ 的存储位置；
 - 2) 给出存储位置为1080的元素下标。

2. 设计算法，计算一个三元组表表示的稀疏矩阵的对角线元素之和。

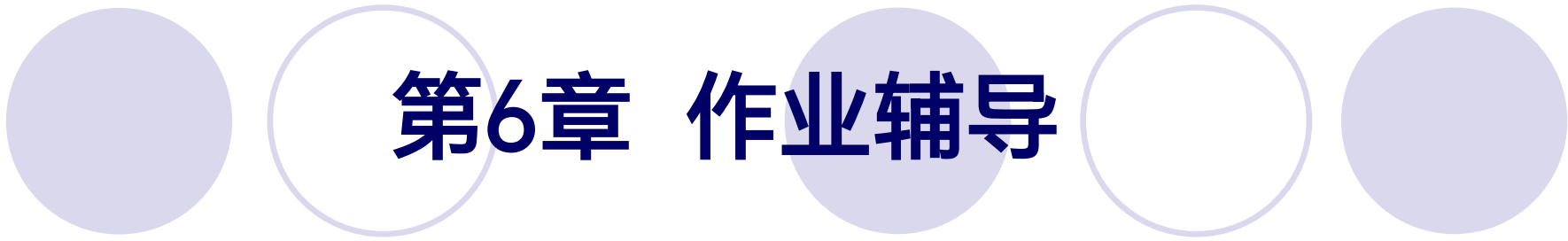
3. 设有三对角矩阵 $A_{n \times n}$ ，将其三条对角线上的元素逐行地存储到向量 $B[0 \dots 3n-3]$ 中，使得 $B[k] = a_{ij}$ ，求：

(1) 用 i, j 表示 k 的下标变换公式。

(2) 用 k 表示 i, j 的下标变换公式。

4. 设二维数组 $A_{5 \times 6}$ 的每个元素占4个字节，已知 $\text{Loc}(a_{00})=1000$ ，A共占多少个字节？A的终端结点 a_{45} 的起始地位为何？按行和按列优先存储时， a_{25} 的起始地址分别为何？

5. 特殊矩阵和稀疏矩阵哪一种压缩存储后会失去随机存取的功能？为什么？



第6章 作业辅导

1. 给出只有3个结点的所有二叉树。
2. 已知二叉树有50个叶子结点，则该二叉树的总结点数至少应该有多少个？
3. 具有 n 个结点的满二叉树的叶子结点的个数是多少？

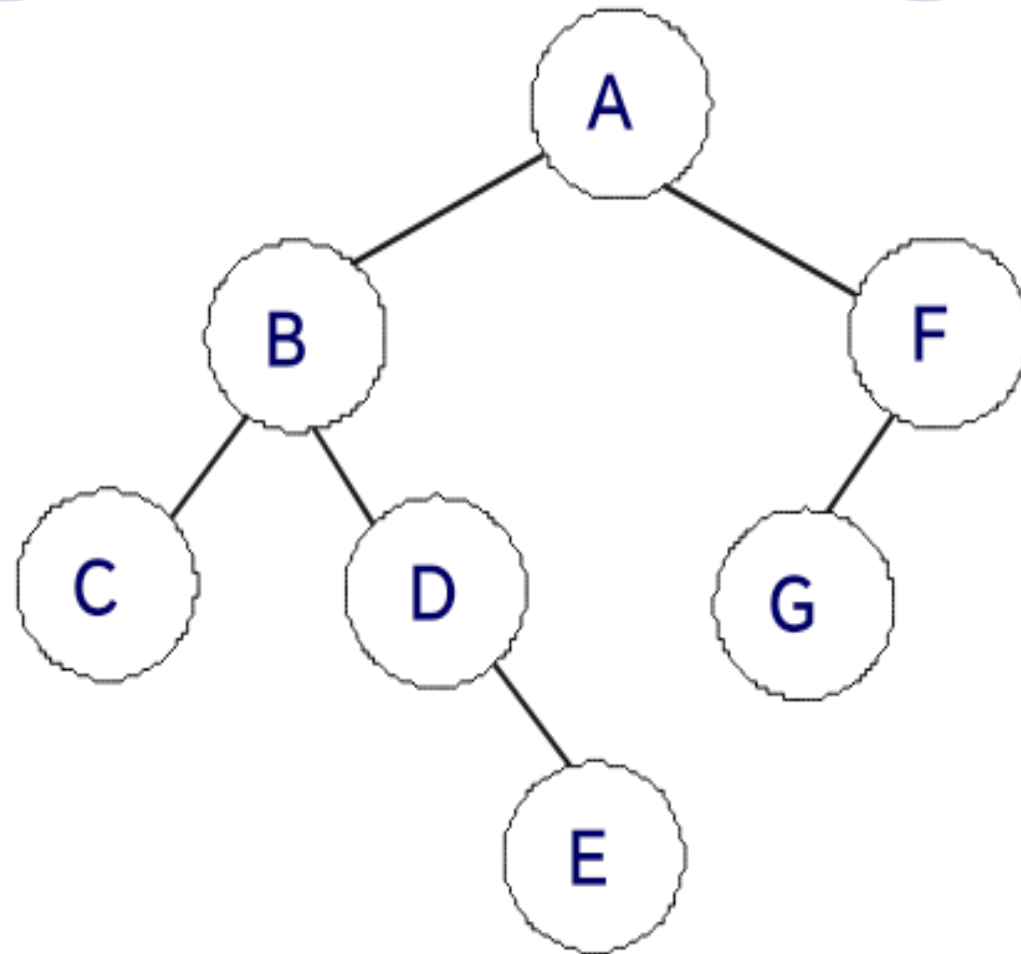
4、已知二叉树的先序、中序和后序序列分别如下，其中有一些看不清的字母用*表示，请构造出一棵符合条件的二叉树并画出该二叉树。

先序序列是：*BC**FG

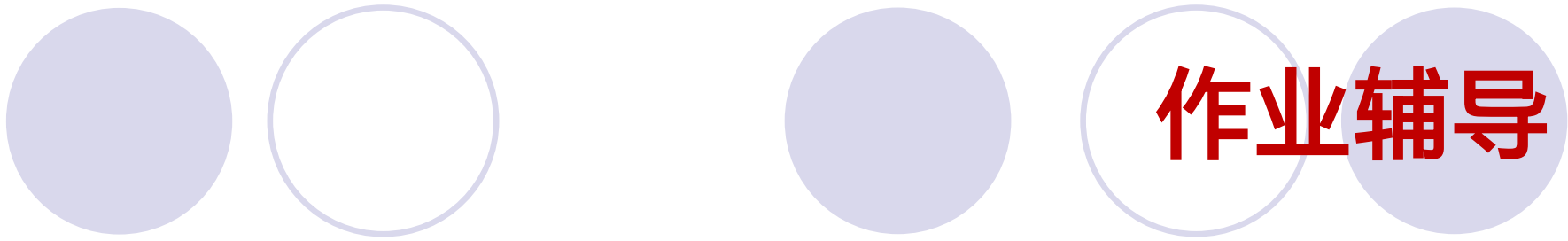
中序序列是：CB*EAG*

后序序列是：*EDB*FA

作业辅导



5、假设二叉树采用二叉链表存储结构存储，设计一个算法，求先序遍历序列中第 k ($1 \leq k \leq$ 二叉树结点个数) 个结点的值。



作业辅导

```
BiTNode searchk(BiTree T,int k)
```

```
{ if ( T ) {
```

```
    count++; // 全局变量，对结点计数
```

```
    if (count==k) return T;
```

```
    searchk( T->lchild, k);
```

```
    searchk(T->rchild, k);
```

```
    } // if
```

```
    return NULL; }
```

6. 以二叉链表为存储结构，分别写出求二叉树结点总数及叶子总数的算法。
7. 以二叉链表为存储结构，写一算法交换各结点的左右子树。

```
void ChangeBinTree(BiTree T) {  
    //交换子树  
    if(T) { //后序遍历  
        BiTNode *temp;  
        ChangeBinTree(T->lchild);  
        ChangeBinTree(T->rchild);  
        temp=T->lchild;  
        T->lchild=T->rchild;  
        T->rchild=temp;  
    }  
}
```

8. 以二叉链表为存储结构，分别写出在二叉树中查找值为 x 的结点及求 x 所在结点在树中层数的算法。

//求层次

```
int InLevel(BiTree T,DataType x){  
    int static L=0; //设一静态变量保存层数  
    if(T) {  
        if(L==0) //若是访问根结点  
        {  
            L++; //第1层
```

if(T->data==x) return L ;

if(T->lchild||T->rchild)

L++; //若根有子树，则层数加1
}

else { //访问子树结点

if(T->data==x) return L ;

if(T->lchild||T->rchild)

L++; //若该结点有子树，则层数加1

else return 0;

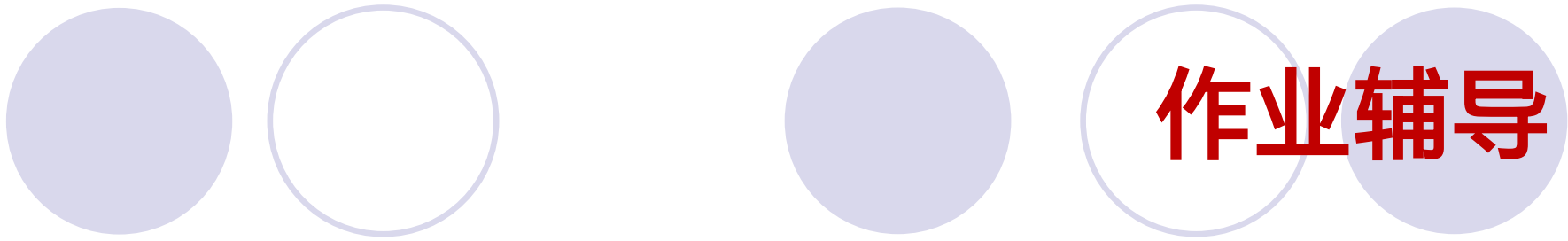
}



作业辅导

```
InLevel(T->lchild,x); //遍历左子树
InLevel(T->rchild,x); //遍历右子树
} // if(T)
}
```

9. 一棵 n 个结点的完全二叉树以向量作为存储结构，试写一非递归算法实现对该树的前序遍历。



作业辅导

/*以向量为存储结构的完全二叉树，其存储在向量中的结点其实是按层次遍历的次序存放的*/

//设结点数据类型为char

typedef char DataType;

#define M 100 //设结点数不超过100

typedef DataType BinTree[M];

```
void Preorder(BinTree T)
```

```
{ //前序遍历算法
```

```
    int n=T[0]; //树中结点数
```

```
    int p[M]; //设置一队列存放结点值
```

```
    int i,j;
```

```
    for(i=1;i<=n;i++) {
```

```
        if (i==1) //根结点
```

```
            j=1;
```

```
        else if(2*j<=n) //左子树
```

作业辅导

```
j=2*j;
```

```
else if(j%2==0&& j<n) //右兄弟
```

```
    j=j+1;
```

```
    else if(j>1) //双亲之右兄弟
```

```
        j=j/2+1;
```

```
p[i]=T[j]; //入队
```

```
printf("%c",p[i]); //打印结点值
```

```
}
```

```
}
```

10. 以线索链表作为存储结构。分别写出在前序线索树中查找给定结点 $*p$ 的前序后继，以及在后序线索树中查找 $*p$ 的后序前趋的算法。

BinThrNode *

SearchPostInPre(BinThrNode *p){

//查找结点*p的前序后继

if(p){

if(p->rtag==0)&&(p->ltag==0)

/*当左、右都为孩子指针，p的前序后继为左孩子*/

return p->lchild;

else return p->rchild; }

}

BinThrNode

*SearchPreInPost(BinThrNode *p)

```
{ //查找*p结点的后序前趋
```

```
  if(p) {
```

```
    if(p->ltag==1||(p->rtag==1)
```

```
      /*当有左线索或无有孩子, p的后续  
      前趋为p->lchild */
```

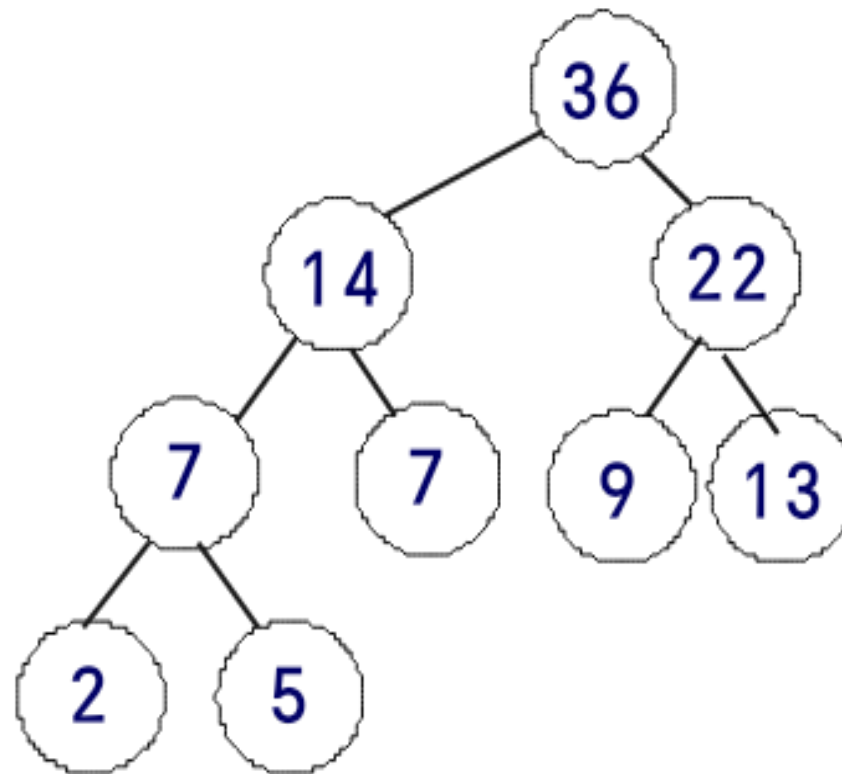
```
        return p->lchild;
```

```
      else return p->rchild;  }
```

```
}
```

11. 以数据集{2, 5, 7, 9, 13}为权值构造一棵哈夫曼树，并计算其带权路径长度。

作业辅导



12. 如下表给出了在一篇有19710个词的英文文章中出现最普通的15个单词的出现频度。假定一篇正文仅由上述字符数据表中的词组成，那么它们的最佳编码是什么？平均长度是多少？

单词及出现的频度

13、已知一棵度为 m 的树中有 n_1 个度为1的结点， n_2 个度为2的结点，... n_m 个度为 m 的结点，问该树中有多少片叶子？

14、高度为 h 的完全二叉树至少有多少个结点？至多有多少个结点？

15、试找出分别满足下面条件的所有二叉树：

- (1)前序序列和中序序列相同；**
- (2)中序序列和后序序列相同；**
- (3)前序序列和后序序列相同；**
- (4)前序、中序、后序序列均相同。**

- 16、高度为 h 的严格二叉树至少有多少个结点?至多有多少个结点?
- 17、在什么样的情况下,等长编码是最优的前缀码?

第7章 图作业辅导

1. 试在无向图的邻接矩阵和邻接链表上实现如下算法：

(1)往图中插入一个顶点

(2)往图中插入一条边

(3)删去图中某顶点

(4)删去图中某条边

```
#define MAX_VERTEX_NUM 30
typedef num{DG,DN,UDG,UDN}GraphKind;
typedef ArcType AdjMtrix
[MAX_VERTEX_NUM][MAX_VERTEX_NUM];
typedef struct {
    VertexType vex[MAX_VERTEX_NUM];
    AdjMtrix arc;
    int vexnum,arcnum;
    GraphKind kind;}GraphMtrix;
```

(1)往图中插入一个顶点

作业辅导

```
AddVertex(GraphMtrix *G,VertexType x)
{ //往无向图的邻接矩阵中插入顶点
    if (G->vexnum>= MAX_VERTEX_NUM )
        Error("顶点数太多");
    else {G->vex[G-> vexnum]=x;
        //将新顶点输入顶点表
        G->vexnum++; //顶点数加1
```



作业辅导

//邻接矩阵增加第 $n+1$ 行，第 $n+1$ 列

for($k=0;k \leq G \rightarrow vexnum;k++$)

$G.arc[k][vexnum]=G.arc[vexnum][k]=0;$

 }**//else**

}

(2)往图中插入一条边

作业辅导

```
Addedge (GraphMtrix *G,VertexType x,  
VertexType y)  
{ //往无向图的邻接矩阵中插入边(x,y)  
    int i,j,k;  
    i=-1;j=-1;  
    for(k=0;k<G->vexnumn;k++)  
        //查找X, Y的编号  
        { if (G->vexs[k]==x) i=k;  
          if (G->vexs[k]==y) j=k;      }  
}
```

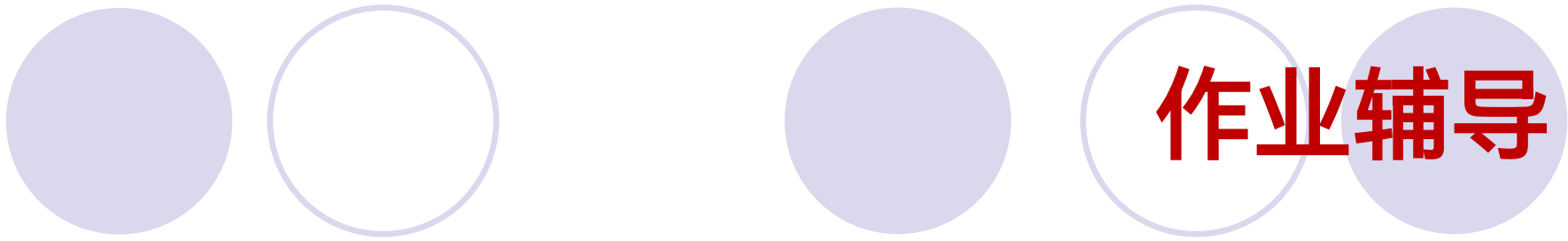
作业辅导

```
if (i==-1||j==-1) Error("结点不存在");  
    else { //插入边(x,y)  
        G->arc[i][j]=1;  
        G->arc[j][i]=1;  
        G->vexarc++; //边数加1  
    }  
}
```

(3)删去图中某顶点

```
void DeleteVertexMGraph(MGraph *G,  
    VertexType x)  
{ //无向图的邻接矩阵中删除顶点x  
    int i,k,j;  
    i=-1;  
    for(k=0;k<G->vexnum;k++)  
        //查找X的编号  
        if (G->vex[k]==x) i=k;  
    if (i==-1) Error("结点不存在");  
    else
```

```
{ //删除顶点以及边
    k=0; //求出与x结点相关联的边数k
    for (j=0;j<G->vexnum;j++)
        if (G->vex[i][j]==1) k++;
        //设置新的边数
    G->arcnum=G->arcnum-k;
    //在邻接矩阵中删除第i行
    for (k=i+1;k<G->vexnum;k++)
        for(j=0;j<G->vexnum;j++)
```

作业辅导

```
G->arc[k-1][j]=G->arc[k][j];  
//在邻接矩阵中删除第i列  
for (k=i+1;k<G->vexnum;k++)  
    for(j=0;j<G->vexnum;j++)  
        G->arc[j][k-1]=G->vexs[j][k];  
G->vexnum--; //总结点数-1  
} //else  
}
```

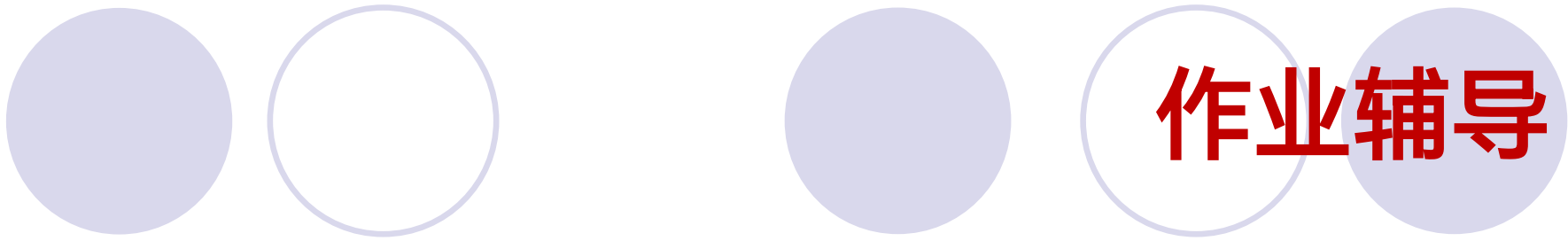
(4)删去图中某条边

作业辅导

```
void DeleteedgeMGraph(MGraph *G,  
    VertexType x,VertexType y)  
{ //无向图的邻接矩阵中删除边(x,y)  
    int i,j,k;  
    i=-1;j=-1;  
    //查找X, Y的编号  
    for(k=0;k<G->vexnum;k++)  
        { if (G->vex[k]==x) i=k;  
          if (G->vex[k]==y) j=k;          }  
}
```

作业辅导

```
if (i==-1||j==-1) Error("结点不存在");  
else if (g->vexs[i][j]==1)  
{ //删除边(x,y)  
  G->arc[i][j]=0;  
  G->arc[j][i]=0;  
  G->arcnum--; //边数减1  
} // else  
}
```



作业辅导

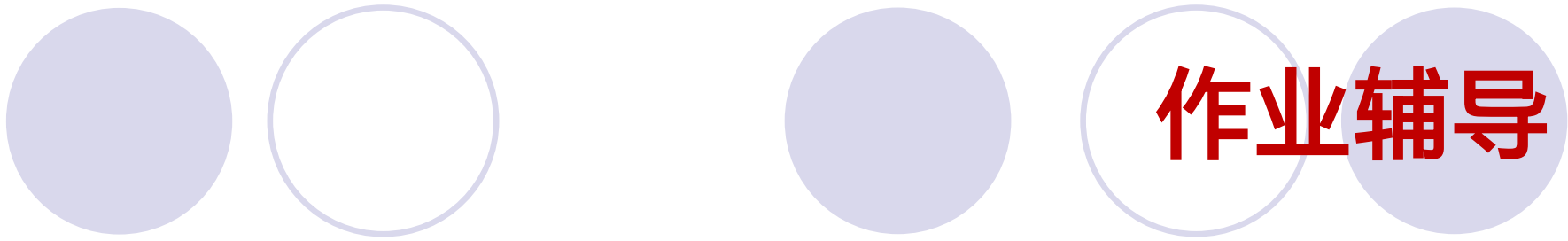
```
#define MAX_VERTEX_NUM 30
```

```
typedef struct EdgeNode{ //边结点
```

```
    int adjvex; //边或弧依赖的顶点
```

```
    struct EdgeNode *next;
```

```
}EdgeNode;
```



作业辅导

```
typedef struct VexNode{ //头结点
    VertexType data;
    EdgeNode *firstedge;
}VexNode,AdjList[MAX_VERTEX_NUM];
typedef struct{ //图
    AdjList vex;
    int vexnum,arcnum;
    int kind;}ALGraph;
```

(1)往图中插入一个顶点

```
void AddVertexALGraPh(ALGrah *G,  
    VertexType x)  
{ //往无向图的邻接表表示中插入一个顶点  
    if (G->vexnum >= MAX_VERTEX_NUM )  
        Error("顶点数太多");  
    else { //将新顶点输入顶点表  
        G->vex[G->vexnum].data=x;  
        //边表置为空表  
        G->vex[G->vexnum].firstedge=NULL;  
        G->vexnum++; //顶点数加1 }    }
```

(2)往图中插入一条边

作业辅导

```
void AddedgeALGraPh(ALGrahp *G,  
    VertexType x,VertexType y)  
{ //往无向图的邻接表中插入边(x,y)  
    int i,j,k;  
    EdgeNode *s,*p;  
    i=-1;j=-1;  
    for(k=0;k<G->vexnum;k++) //查找X, Y的  
        编号  
        { if (G->vex[k].data==x) i=k;  
          if (G->vex[k].data==y) j=k;      }  
}
```



作业辅导

```
if (i==-1||j==-1) Error("结点不存在");
else {
    s=G->vext[i].firstedge;
    //查看邻接表中有无(x,y)
    while ((s)&&(s->adjvex!=j)
        s=s->next;
    if (!s) //当邻接表中无边(x,y),插入边(x,y)
    { //生成边表结点
p=(EdgeNode *)malloc(sizeof(EdgeNode));
```


作业辅导

```
p->adjvex=j; //邻接点序号为j  
//将新结点*s插入顶点x的边表头部
```

```
p->next=G->vex[i].firstedge;
```

```
G->vex[i].firstedge=p;
```

```
p=(EdgeNode *)malloc(sizeof(EdgeNode));
```

```
p->adjvex=i; //邻接点序号为i
```

```
//将新结点*s插入顶点x的边表头部
```

```
p->next=G->vex[j].firstedge;
```

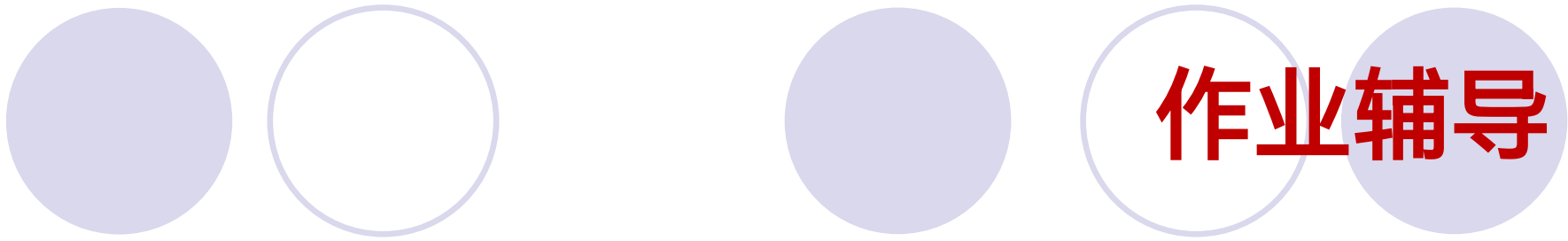
```
G->vex[j].firstedge=p;
```

```
G->arcnum++; //边数加1 } } }
```

(3)删去图中某顶点

作业辅导

```
void DeleteVertexALGraPh(ALGrahph *G,  
    VertexType x)  
{ //无向图的邻接表中删除顶点x  
    int i,k,j;  
    EdgeNode *s,*p,*q;  
    i=-1;  
    for(k=0;k<G->vexnum;k++) //查找X的编号  
        if (G->vex[k].data==x) i=k;  
    if (i==-1) Error("结点不存在");  
    else { //删除与x相关联的边
```



作业辅导

```
s=G->vex[i].firstedge;
```

```
while (s) {
```

```
//删除与i相关联的其他结点边表中表结点
```

```
p=G->vex[s->adjvex].firstedge;
```

```
if ((p)&&(p->adjvex==i) )//是第一个边表结点，修改头指针{
```

```
G->vex[s->adjvex].firstedge=p->next;
```

```
free(p);      } //if
```

作业辅导

```
else //不是第一个 边表结点, 查找并删除
{while( (p)&&(p->next)&&(p->next->
adjvex!=i))
    p=p->next;
q=p->next;
p->next=q->next; free(q);    } //else
//在i结点的边表中删除表结点

q=s;s=s->next;free(q);
G->arcnum--; //边数减1    } //while
```

//调整顶点表

```
for (j=i;j<G->vexnum-1;j++){  
    G->vex[j].firstedge=G->vex[j+1].firstedge;  
    G->vex[j].data=G->vex[j+1].data;  
} //for  
G->vexnum--; //顶点数减1  
} //else  
}
```

(4)删去图中某条边

作业辅导

```
void DeleteedgeALGraPh(ALGraph *G,  
    VertexType x,VertexType y)  
{ //往无向图的邻接表中删除边(x,y)  
    int i,j,k;  
    EdgeNode *s,*p;  
    i=-1;j=-1;  
    for(k=0;k<G->n;k++) //查找X, Y的编号  
    { if (G->vexk].data==x) i=k;  
      if (G->vex[k].data==y) j=k;    }  
    if (i==-1||j==-1) Error("结点不存在");  
    else {
```

作业辅导

//在i的边表中删除值为j的边表结点

```
s=G->vex[i].firstedge;
```

```
if ((s)&&(s->adjvex==j) ){
```

//是第一个边表结点，修改头指针

```
G->vex[i].firstedge=s->next;
```

```
free(s);      }
```

```
else{ //不是第一个 边表结点，查找并删除
```

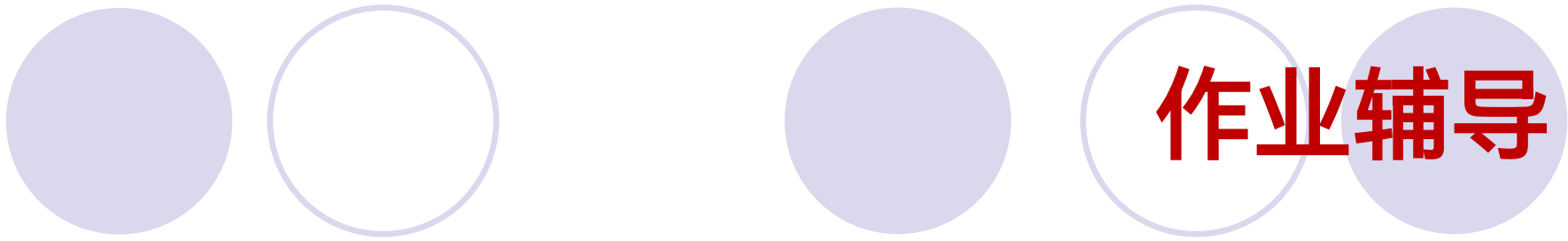
```
while ((s)&&(s->next)&&(s->next->  
adjvex!=j))    s=s->next;
```



作业辅导

```
if (!s->next)    Error("无此边");
else {
    p=s->next; s=p->next; free(p);    }
}

//在j的边表中删除值为i的边表结点
s=G->vex[j].firstedge;
if( (s)&&(s->adjvex==i)) {
    G->vex[j].firstedge=s->next;    free(s); }
```

作业辅导

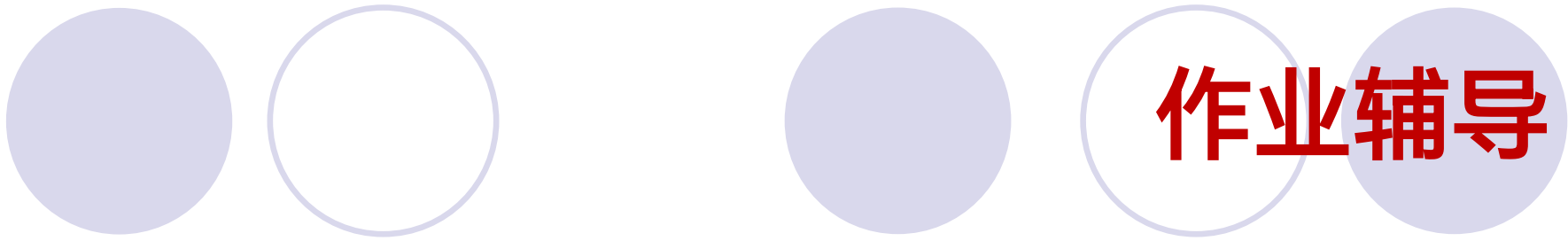
```
else{  
    while ((s)&&(s->next)&&(s->next->adjvex  
!=j))        s=s->next;  
    p=s->next;s=p->next;free(p);        }  
    G->arcnum--;  
}  
}
```

2. 试以邻接表和邻接矩阵为存储结构，分别写出基于DFS和BFS遍历的算法来判别顶点 v_i 和 $v_j (i \neq j)$ 之间是否有路径。

(1)基于采用邻接矩阵的DFS

作业辅导

```
int pathDFSM(MatrixGraph *G, int i,int j)
{ /*以邻接矩阵为存储结构，判断vi和vj之间是
  否有路径，若有返回1，否则返回0*/
  int k;
  visited[i]=1;
  for(k=0; k<G->vexnum; k++)
    //依次搜索vi的邻接点
    if(G->arc[i][k]==1&&!visited[k])
```



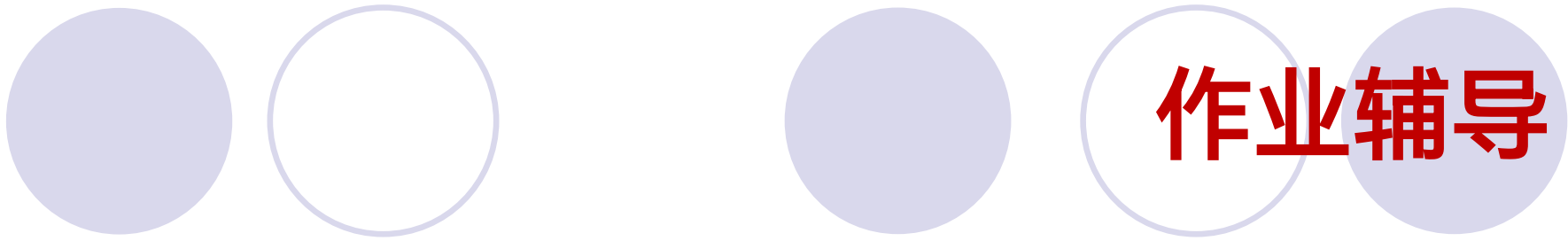
作业辅导

```
if (k==j) return 1; //有路径相通
else
    return(pathDFSM(G,k,j);
return 0; //无路径相通
} //DFSM
```

(2)基于采用邻接表的DFS

作业辅导

```
int PATHDFS(ALGraph *G, int i,int j){  
    /*以邻接表为存储结构，判断vi和vj之间是否  
    有路径，若有返回1，否则返回0*/  
    EdgeNode *p;  
    visited[i]=1; //标记vi已访问  
    //取vi边表的头指针  
    p=G->vertices[i].firstedge;  
    while(p){ /*依次搜索vi的邻接点vk，这  
    里k=p->adjvex*/
```



作业辅导

```
if (!visited[p->adjvex]) //若vk尚未被访问
    if (p->adjvex==j)      return 1;
    else //则以Vk为出发点向纵深搜索
        return PATHDFS(G, p->adjvex,j);
p=p->next; //找vi的下一邻接点
}
return 0;
} //PATHDFS
```

(3)基于邻接矩阵的BFS算法

作业辅导

```
int pathBFSS(MGraph *G, int i,int j){  
    /*以邻接矩阵为存储结构，判断vi和vj之间是  
    否有路径，若有返回1，否则返回0*/  
    int k;  
    CirQueue Q;  
    //须将队列定义中DataType改为int  
    initQueue(&Q); //队列初始化  
    visited[i]=1;  
    EnQueue(&Q, i);  
    while(!QueueEmpty(&Q)){
```



作业辅导

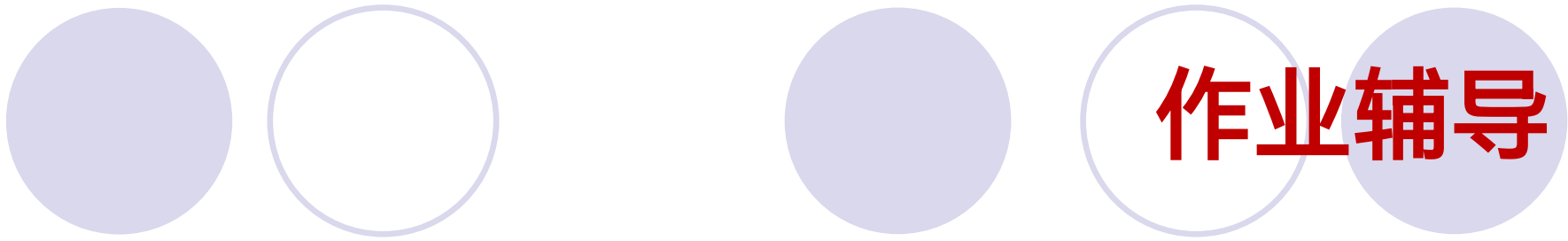
```
i=DeQueue(&Q); //vi出队
for(k=0;k<G->vexnum;k++)
    //依次搜索vi的邻接点vk
    if(G->arc[i][k]==1&&!visited[k]){
        //vk未访问
        if (k==j) return 1; visited[k]=1;
        EnQueue(&Q, k); //访问过的vk入队
    } //endwhile
return 0;    } //pathBFSM
```


(4)基于邻接表为存储结构的BFS算法

```
int BFS(ALGraph *G, int i,int j){  
    /*以邻接表为存储结构，判断vi和vj之间是否  
    有路径，若有返回1，否则返回0*/  
    int i;  
    CirQueue Q;  
    EdgeNode *p;  
    InitQueue(&Q);  
    visited[i]=1;  
    EnQueue(&Q, i); //vi已访问，将其入队  
    。 （实际上是将其序号入队）
```

作业辅导

```
while(!QueueEmpty(&Q)){ //队非空则执行
    i=DeQueue(&Q); //相当于vi出队
    //取vi的边表头指针
    p=G->adjlist[i].firstedge;
    while(p){ //依次搜索vi的邻接点vk(令p->
        adjvex=k)
        if(!visited[p->adjvex]) //若vk未访问过
            if (p->adjvex==j)        return 1;
            else{
```



作业辅导

```
visited[P->adjvex]=1;  
EnQueue(&Q, p->adjvex);  
} //访问过的vk入队  
    p=p->next; //找vi的下一邻接点  
} //endwhile  
return 0;  
} //end of pathBFS
```



作业辅导

3. 试分别写出求DFS和BFS生成树(或生成森林)的算法，要求打印出所有的树边。

(1)求DFS生成树

作业辅导

```
typedef enum{FALSE, TRUE}Boolean;  
//FALSE为0, TRUE为1
```

```
Boolean visited[MaxVertexNum];
```

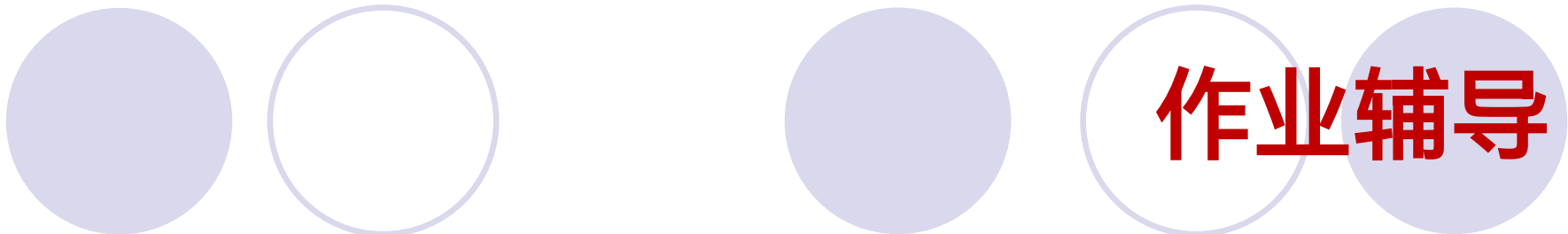
```
//访问标志向量是全局量
```

```
void DFSTraverseTREE(ALGraph *G){
```

```
//求深度优先生成树(以邻接表表示的图G),  
而以邻接矩阵表示G时, 算法完全与此相同
```

```
int i;
```

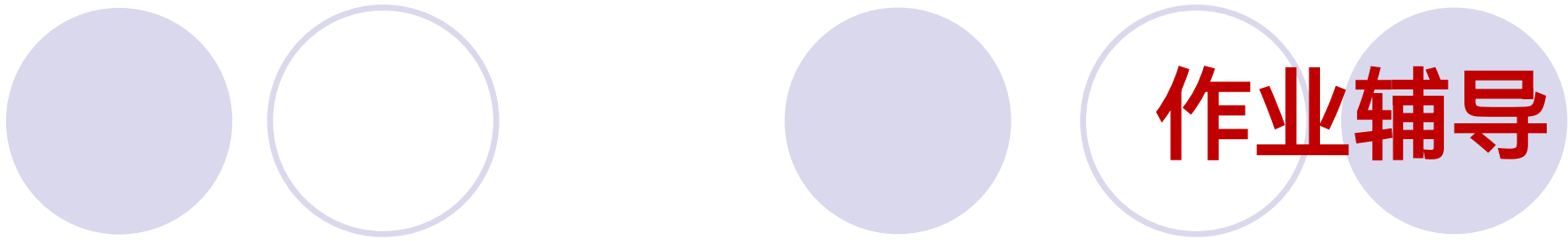
```
for(i=0;i<G->vexnun;i++)
```



作业辅导

```
visited[i]=FALSE; //标志向量初始化
for(i=0;i<G->vexnum; i++)
    if(!visited[i]) //vi未访问过
        DFSTree(G, i); //以vi为源点开始
DFS搜索,求DFS生成树的边
} //DFSTraverse

void DFSTree(ALGraph *G, int i){
    //以vi为出发点对邻接表表示的图G进行深度
    优先搜索,打印生成树(生成森林)的边
}
```



作业辅导

```
EdgeNode *p;  
visited[i]=TRUE; //标记vi已访问  
p=G->vex[i].firstedge; //取vi边表的头指针  
while(p){ //依次搜索vi的邻接点vj, 这里j=p->  
    adjvex  
    if (!visited[p->adjvex]) //若vj尚未被访问  
    { printf("(%c,%c)\n",G->vex[i].data,G->  
vex[p->adjvex].data) ;
```

作业辅导

```
DFSTree(G, p->adjvex); //则以Vj为出发点  
    向纵深搜索      }  
    p=p->next; //找vi的下一邻接点  }  
} //DFS
```

```
void DFSMTree(MGraph *G, int i){  
    /*以vi为出发点对邻接矩阵表示的图G进行深  
    度优先搜索,打印生成树(生成森林)的边*/  
    int j;  
    visited[i]=TRUE;
```




作业辅导

```
for(j=0; j<G->vexnum; j++)  
    //依次搜索vi的邻接点  
    if(G->arc[i][j]==1&&!visited[j]){  
        printf("(%c,%c)\n",G->vex[i],G->  
vex[j]); // 打印边  
DFSMTTree(G, j); /*(vi, vj) ∈ E, 且vj未访问  
过, 故vj为新出发点*/  
}  
} //DFSMTTree
```

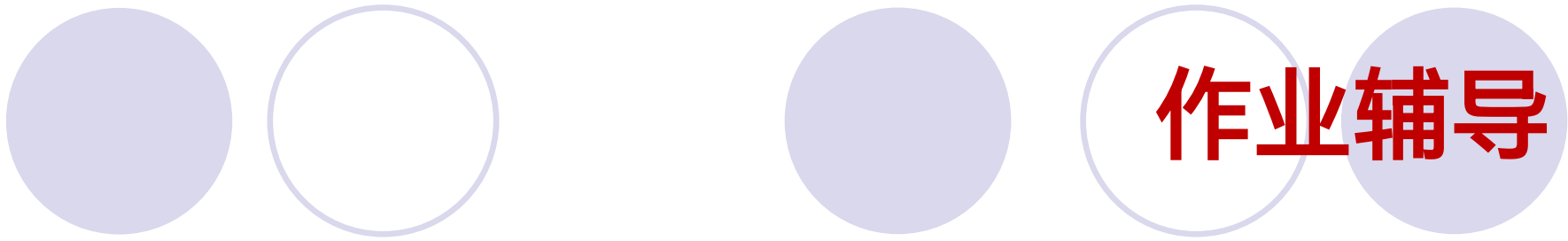
(2)求BFS生成树

```
typedef enum{FALSE, TRUE}Boolean;  
//FALSE为0, TRUE为
```

```
Boolean visited[MaxVertexNum]; //访问  
标志向量是全局量
```

```
void BFS_Traverse_TREE(ALGraph *G){
```

```
/*求广度优先生成树(以邻接表表示的图G),  
而以邻接矩阵表示G时, 算法完全与此相同  
只要将BFSTree(G, i)改为BFSTMTree(G,  
i)即可*/
```



作业辅导

```
int i;  
for(i=0;i<G->vexnum;i++)  
    visited[i]=FALSE; //标志向量初始化  
for(i=0;i<G->vexnum; i++)  
    if(!visited[i]) //vi未访问过  
        BFSTree(G, i); /*以vi为源点开始BFS  
搜索,求BFS生成树的边*/  
} //BFSTraverse
```



作业辅导

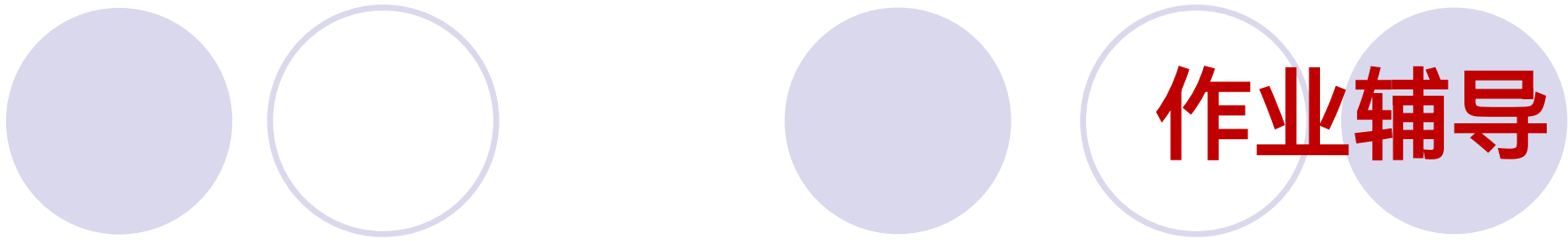
```
void BFSTree(ALGraph*G, int k) {  
    /* 以vk为源点对用邻接表表示的图G进行广  
       度优先搜索,并求出BFS生成树边*/  
    int i;  
    CirQueue Q; /*须将队列定义中DataType  
                  改为int*/  
    EdgeNode *p;  
    InitQueue(&Q); //队列初始化  
    visited[k]=TRUE;
```

作业辅导

```
EnQueue(&Q, k); //vk已访问，将其入队
。 (实际上是将其序号入队 while(!
QueueEmpty(&Q)){
//队非空则执行
i=DeQueue(&Q); //相当于vi出队
p=G->vex[i].firstedge; //取vi的边表头指
针
while(p){ //依次搜索vi的邻接点vj(令p->
adjvex=j)
```

作业辅导

```
if(!visited[p->advex]){ //若vj未访问过
    printf("(%c,%c)\n", G->vex[i].data,G->
    vex[p->advex].data); //打印边
    visited[P->advex]=TRUE;
    EnQueue(&Q, p->advex); //访问过的vj
    入队 } //endif
    p=p->next; //找vi的下一邻接点
} //endwhile } //endwhile
} //end of BFSTree
```



作业辅导

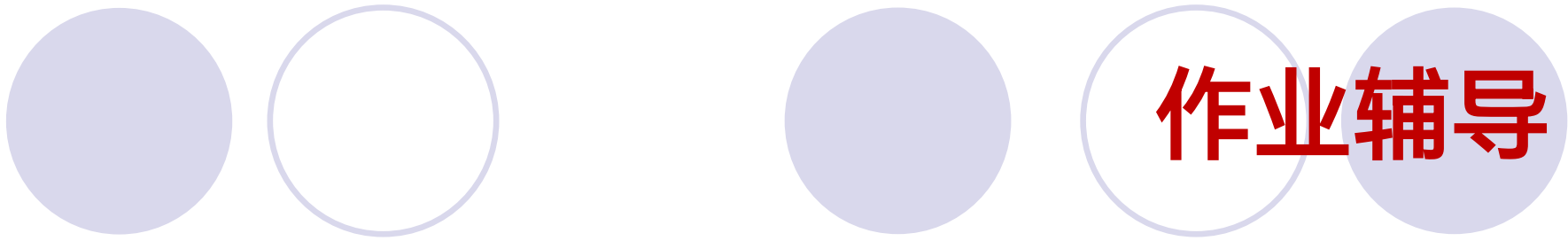
```
void BFSMTree(MGraph *G, int k) {  
    // 以vk为源点对用邻接矩阵表示的图G进行  
    广度优先搜索,并求出BFS生成树边  
    int i, j;  
    CirQueue Q;  
    InitQueue(&Q);  
    visited[k]=TRUE;  
    EnQueue(&Q, k);  
    while(!QueueEmpty(&Q)){
```



作业辅导

```
i=DeQueue(&Q); //vi出队
for(j=0;j<G->n;j++) //依次搜索vi的邻接点vj
    if(G->ara[i][j]==1&&!visited[j]){
        //vi未访问, 打印边
        printf("(%c,%c)\n",G->vex[i],G->vex[j])
        visited[j]=TRUE;
        EnQueue(&Q, j); //访问过的vj入队
    } //endwhile
} //BFSMTree
```


4. 写一算法求连通分量的个数并输出各连通分量的顶点集。

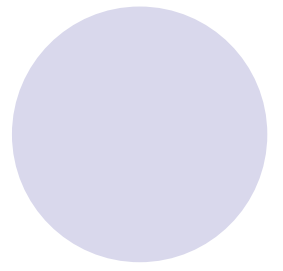
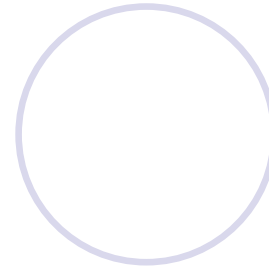
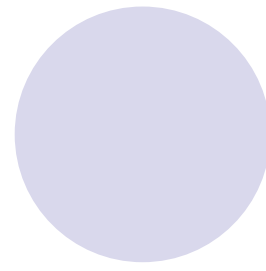
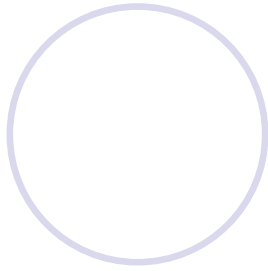
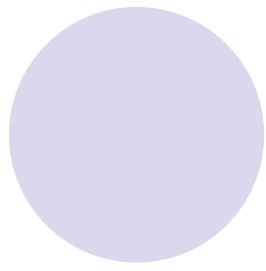


作业辅导

```
typedef enum{FALSE, TRUE}Boolean;  
//FALSE为0, TRUE为1  
    Boolean visited[MaxVertexNum]; //访  
问标志向量是全局量  
    void DFSTraverse(ALGraph *G)  
    { //深度优先遍历以邻接表表示的图G,  
求连通分量的个数和各连通分量的顶点集  
        int i;  
        for(i=0;i<G->n;i++)  
            visited[i]=FALSE; //标志向量初始  
化
```

5. 设图中各边的权值都相等，试以邻接矩阵和邻接表为存储结构，分别写出算法：
- (1)求顶点 v_i 到顶点 $v_j(i \neq j)$ 的最短路径
 - (2)求源点 v_i 到其余各顶点的最短路径
- 要求输出路径上的所有顶点(提示：利用BFS遍历的思想)

6. 以邻接表为存储结构，写一个基于DFS遍历策略的算法，求图中通过某顶点 v_k 的简单回路(若存在)。



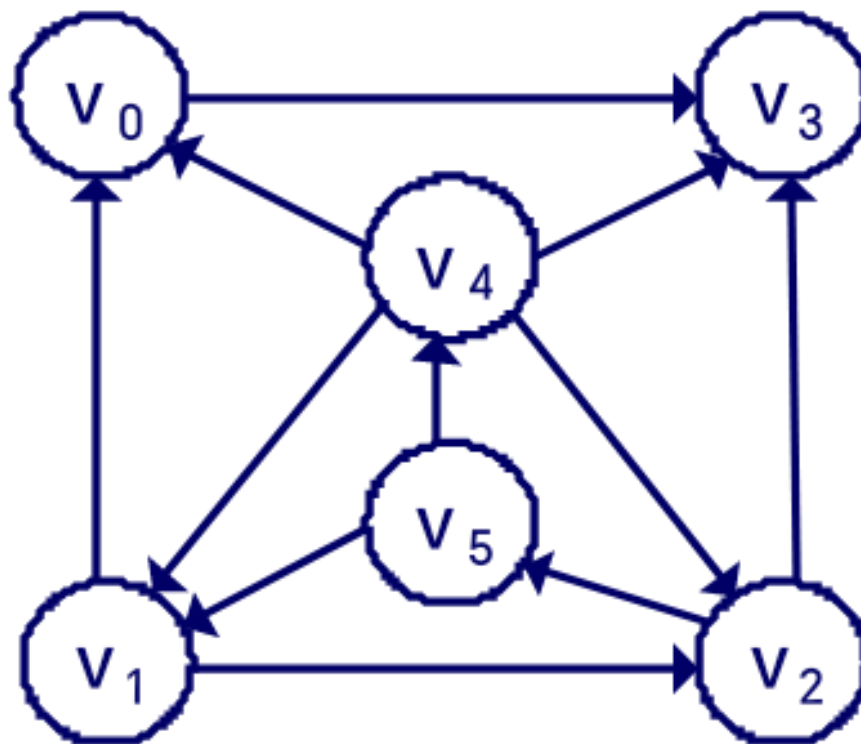
```
int circleDFS(ALGraph *G, int k){  
    //以vk为出发点对邻接表表示的图G,求  
    简单回路,若存在返回1, 否则返回0  
    EdgeNode *p;  
    printf("%c", G->adjlist[k].vertex); //  
    访问顶点vk  
    visited[k]=TRUE; //标记vk已访问  
    p=G->adjlist[k].firstedge; //取vk边表  
    的头指针  
    while(p){//依次搜索vk的邻接点vj, 这里  
    j=p->adivex
```

7. 设有向图G如下图1所示，试给出：

- (1) 该图的邻接矩阵；
 - (2) 该图的邻接表；
 - (3) 该图的逆邻接表；
 - (4) 从V0出发的“深度优先”遍历序列；
 - (5) 从V0出发的“广度优先”遍历序列；
- 

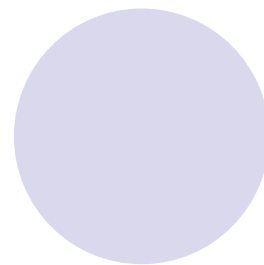
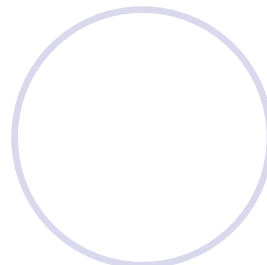
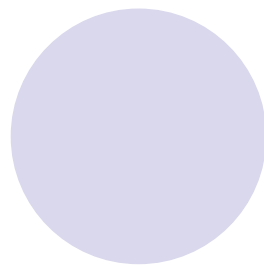
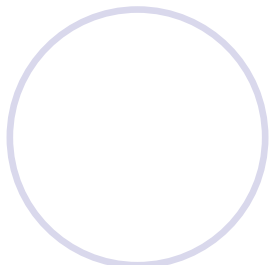
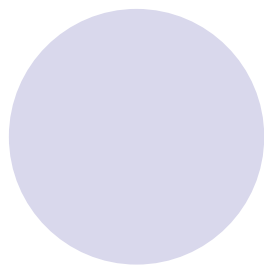
作业辅导

(6) 画出从 V_0 出发的深度优先生成树和广度优先生成树。



8. 解答下面的问题：

- (1) 求网的最小生成树有哪些算法？各适用何种情况？为什么？
- (2) 有如下的网络邻接矩阵，画出一棵最小生成树。



∞	17	∞	∞	20	22
17	∞	6	7	∞	12
∞	6	∞	11	∞	∞
∞	7	11	∞	19	15
20	∞	∞	19	∞	34
22	12	∞	15	34	∞

9. 有一带权无向图的顶点集合为 $\{v_0, v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$ 。已知其邻接矩阵的三元组表示如下所示。

- (1) 画出该无向图的邻接表。
- (2) 画出所有可能的最小生成树。

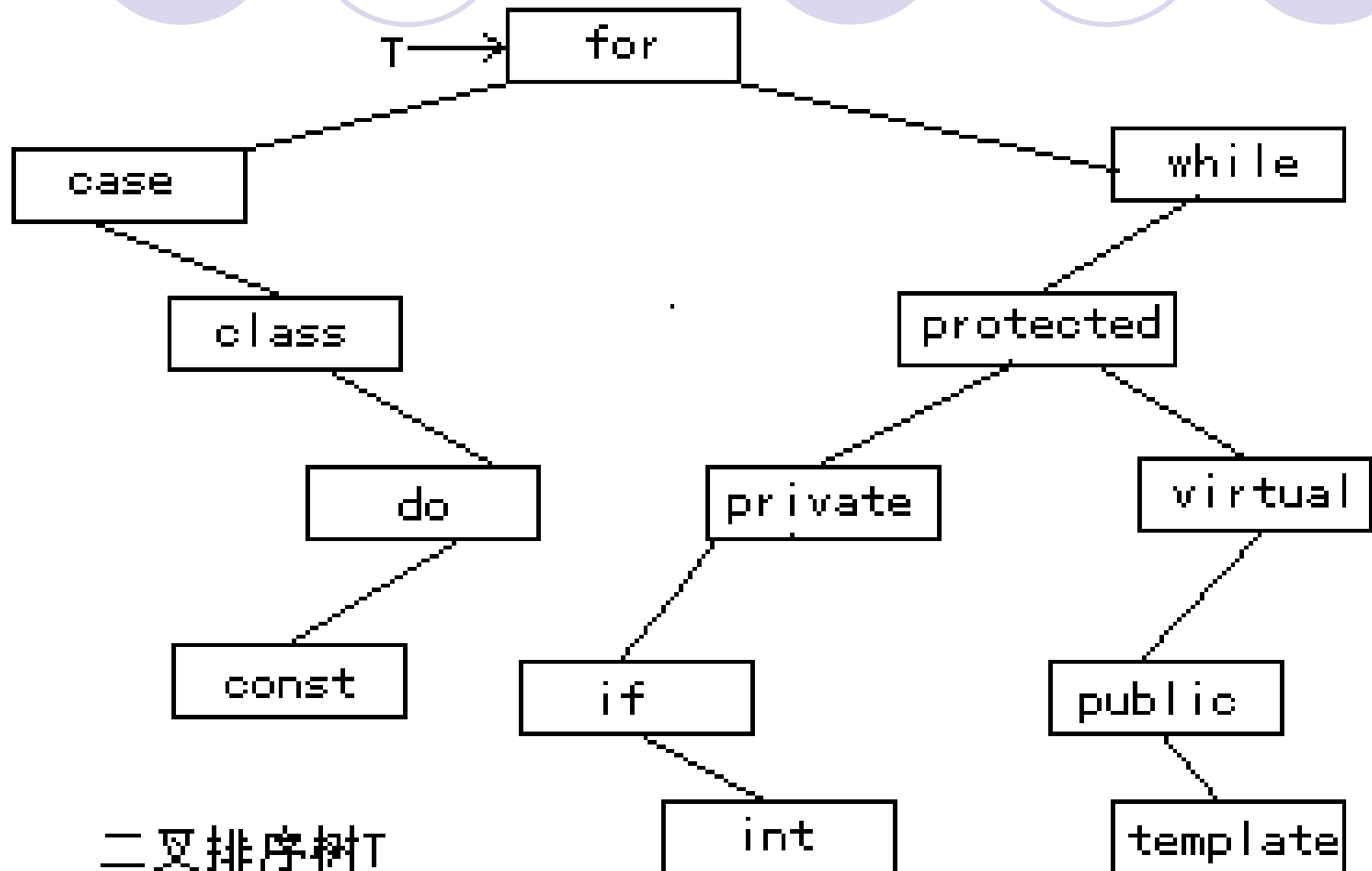
- (3) 根据 (1) 所得的邻接表分别写出从 v_0 出发进行深度优先遍历和广度优先遍历序列。**
- (4) 求出从 v_0 到其余个顶点的最短路经。**
- (5) 求出各顶点对之间的最短路经。**

1. 若对具有 n 个元素的有序的顺序表和无序的顺序表分别进行顺序查找，试在下述两种情况下分别讨论两者在等概率时的平均查找长度：
 - (1) 查找不成功，即表中无关键字等于给定值 K 的记录；
 - (2) 查找成功，即表中有关关键字等于给定值 K 的记录。

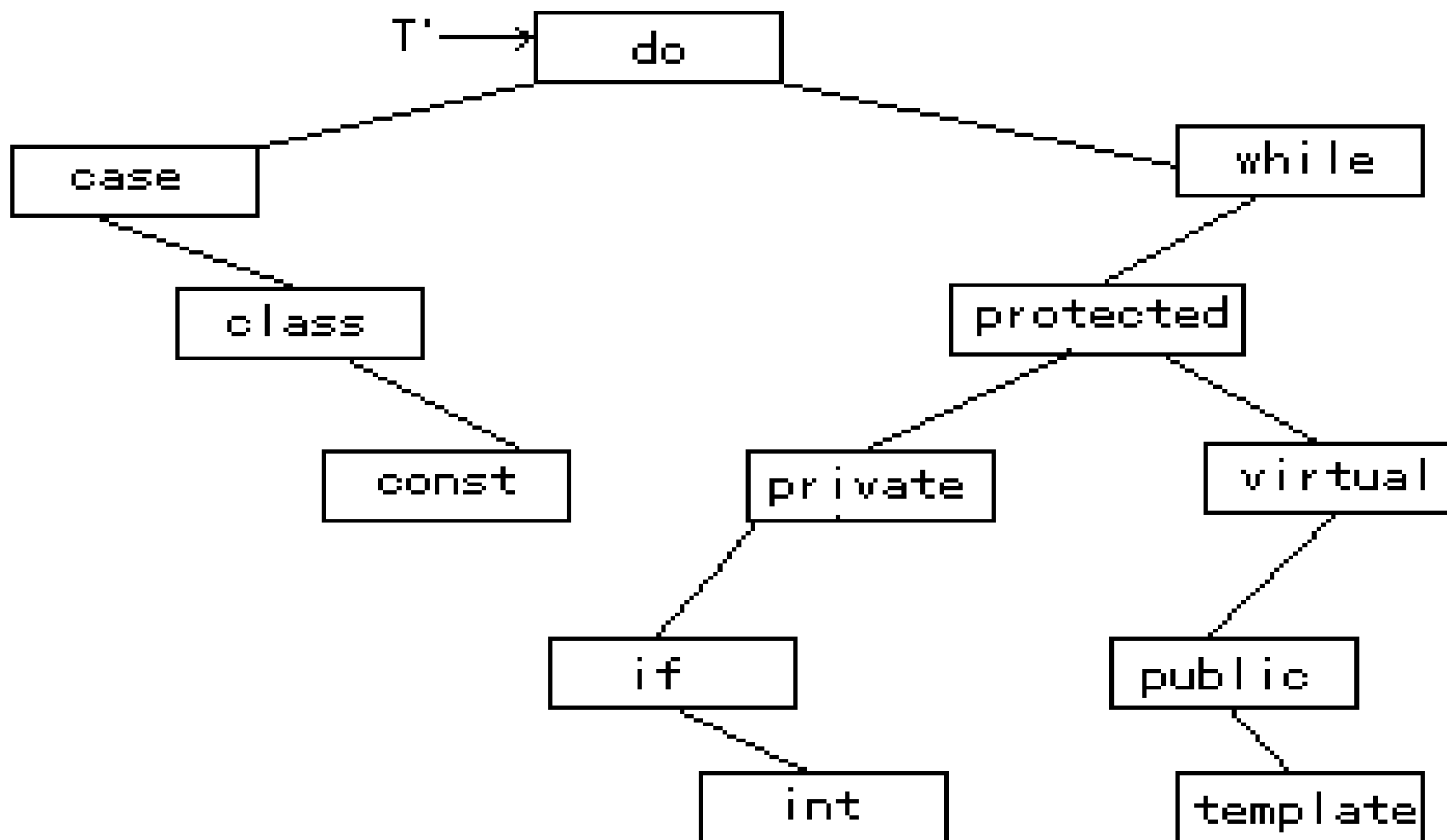
2. 试分别画出在线性表 (a,b,c,d,e,f,g) 中进行折半查找，以查关键字等于e,f和h的过程。
3. 画出对长度为18的有序的顺序表进行二分查找的判定树，并指出在等概率时查找成功的平均查找长度，以及查找失败时所需的最多的关键字比较次数。

4. 将(for, case, while, class, protected, virtual, public, private, do, template, const, if, int)中的关键字依次插入初态为空的二叉排序树中，请画出所得到的树T。然后画出删去for之后的二叉排序树T'，若再将for 插入T'中得到的二叉排序树T''是否与T相同？最后给出T''的先序、中序和后序序列。

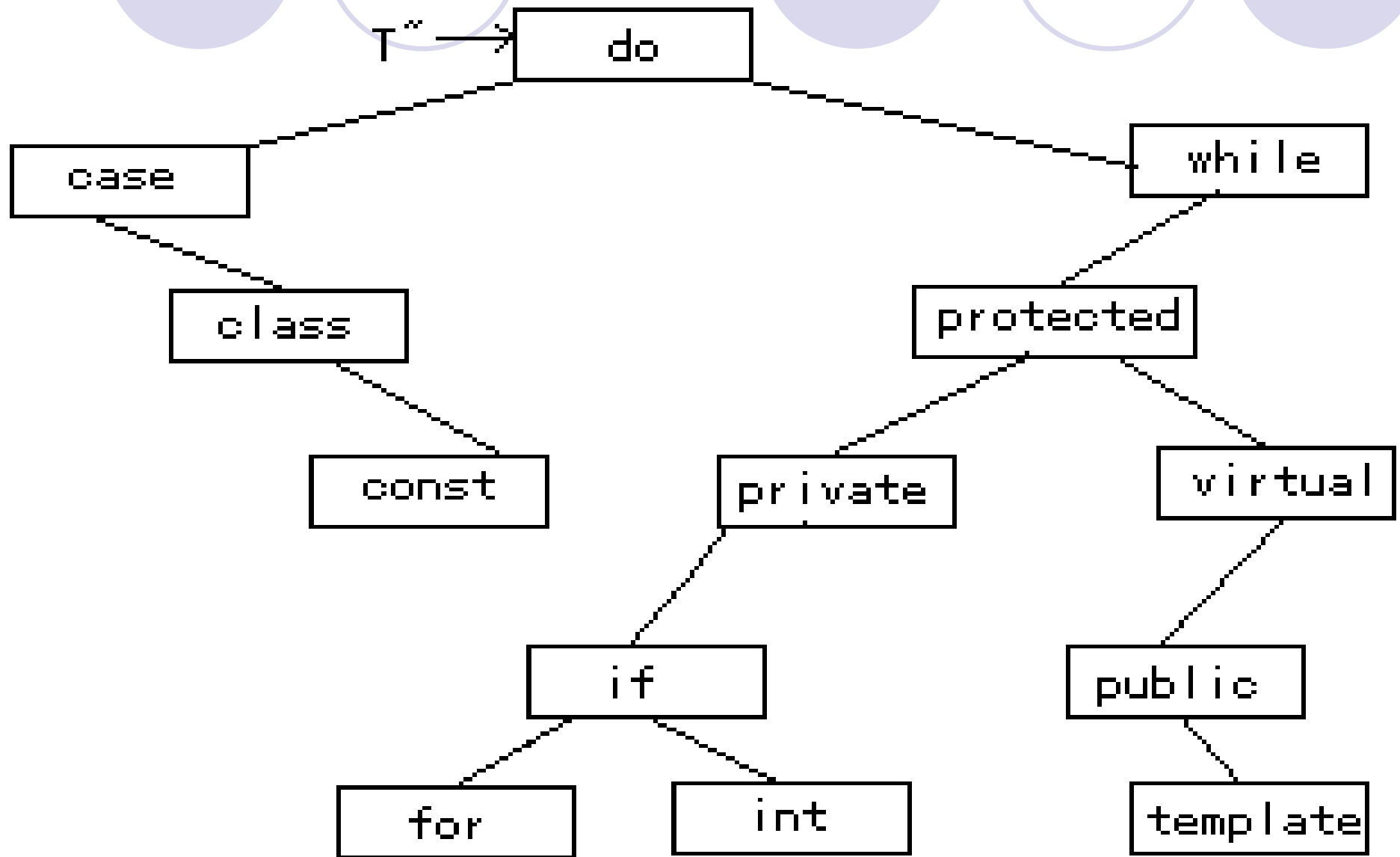
二叉排序树T如下图:



删去for后的二叉排序树如下图:



再插入结点for后的二叉排序树T'':



5. 对给定的关键字集合，以不同的次序插入初始为空的树中，是否有可能得到同一棵二叉排序树？
6. 将二叉排序树T的先序序列中的关键字依次插入一空树中，所得和二叉排序树T'与T否相同？为什么？

作业辅导

7. 设二叉排序树中关键字由1至1000的整数构成，现要查找关键字为363的结点，下述关键字序列哪一个不可能是在二叉排序树上查找到的序列
- (a) 2, 252, 401, 398, 330, 344, 397, 363;
;
 - (b) 924, 220, 911, 244, 898, 258, 362, 363;
 - (c) 925, 202, 911, 240, 912, 245, 363;
 - (d) 2, 399, 387, 219, 266, 382, 381, 278, 363.

- 1、散列存储与其它存储主要有什么区别？
- 2、发生冲突都有哪些解决冲突的办法？
- 3、已知关键字序列为(PAL,LAP,PAM,MAP, PAT,PET,SET,SAT,TAT,BAT)试为它们设计一个散列函数，将其映射到区间 $[0..n-1]$ 上，要求碰撞尽可能的少。这里
 $n=11,13,17,19$.

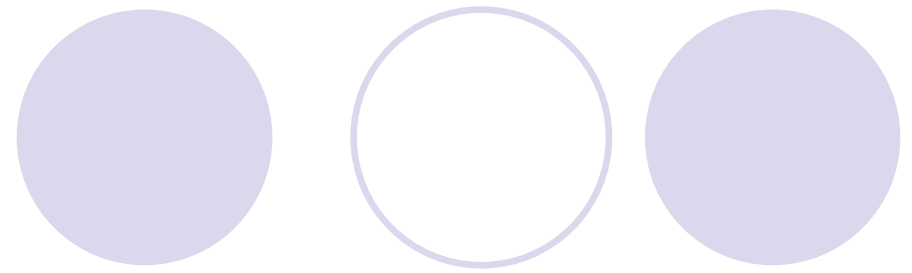
4、对于一组给定的、固定不变的关键字序列，有可能设计出无冲突的散列函数 H ，此时称 H 为完备的散列函数(perfect hashing function)，若 H 能无冲突地将关键字完全填满散列表，则称 H 是最小完备(minimal perfect)的散列函数。通常找完备的散列函数非常困难，找最小完备的散列函数就更困难。请问：

作业辅导

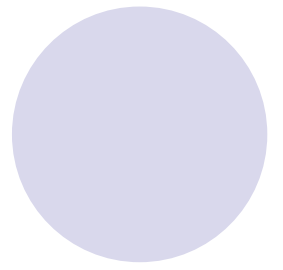
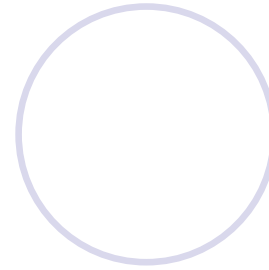
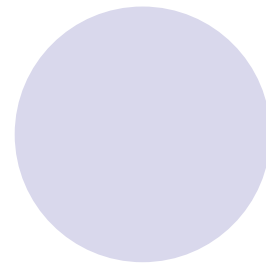
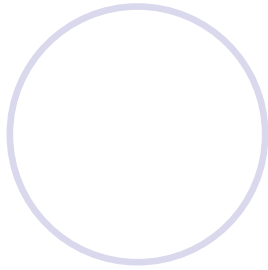
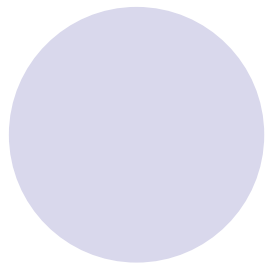
- (1) 若 h 是已知关键字集合 K 的完备的散列函数，若要增加一个新的关键字到集合 K ，一般情况下 H 还是完备的吗？
- (2) 已知关键字集合为(81, 129, 301, 38, 434, 216, 412, 487, 234)，散列函数为 $H(x)=(x+18)/63$ ，请问 H 是完备的吗？它是最小完备的吗？
- (3) 考虑由字符串构成的关键字集合(Bret, Jane, Shirley, Bryce, Michelle, Heather)，试为散列表[0..6]，设计一个完备的散列函数。(提示：考虑每个字符串的第3个字符，即 $s[2]$)

6、设散列表长度为11，散列函数 $h(x)=x\%11$ ，给定的关键字序列为：1，13，13，34，38，33，27，22.试画出分别用拉链法和线性探查法解决冲突时所构造的散列表，并求出在等概率情况下，这两种方法查找成功和失败时的平均查找长度。请问装填因子的值是什么？

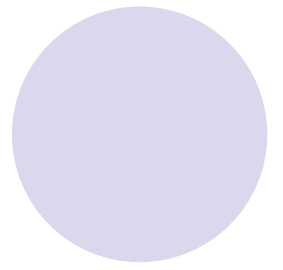
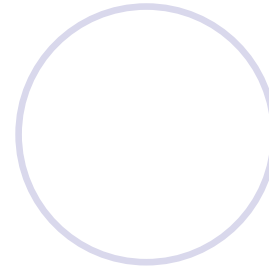
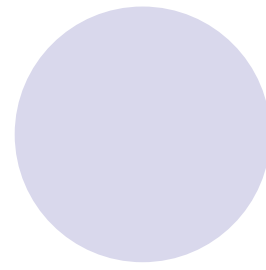
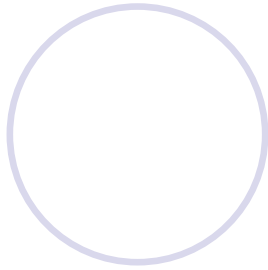
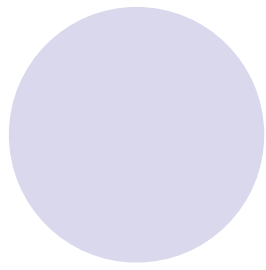
45页, 71页, 33



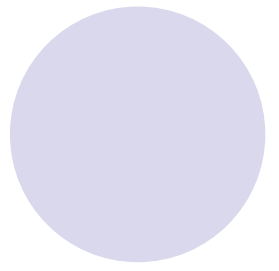
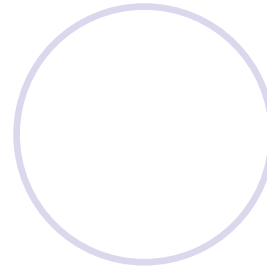
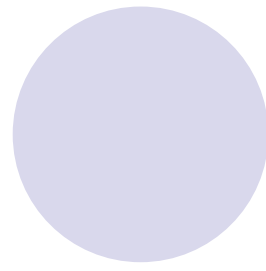
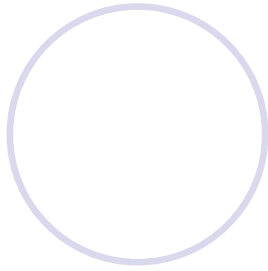
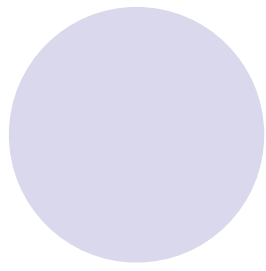
1. 比较直接插入排序和希尔排序的不同点。
2. 给出关键字序列{4, 5, 1, 2, 8, 6, 7, 3, 10, 9}的直接插入排序过程。
3. 给出关键字序列{4, 5, 1, 2, 8, 6, 7, 3, 10, 9}的希尔排序过程。
4. 指出堆和二叉排序树的区别。
5. 给出关键字序列{4, 5, 1, 2, 8, 6, 7, 3, 10, 9}的冒泡排序过程。



6. 设计算法，实现双向冒泡排序算法，即在排序过程中交替改变扫描方向。
7. 给出关键字序列{4, 5, 1, 2, 8, 6, 7, 3, 10, 9}的二路归并排序过程。
8. 给出关键字序列{4, 5, 1, 2, 8, 6, 7, 3, 10, 9}的基数排序过程。
9. 将哨兵放在 $R[n]$ 中，被排序的记录放在 $R[0..n-1]$ 中，重写直接插入排序算法。

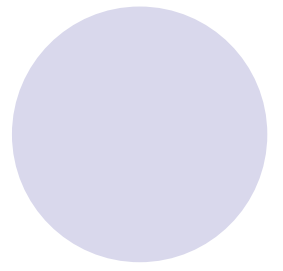
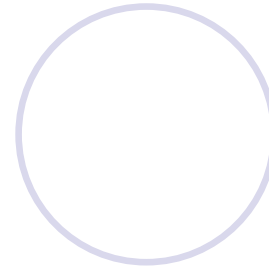
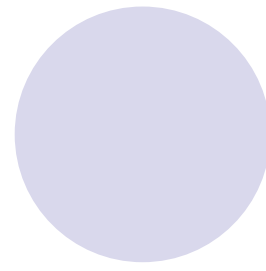
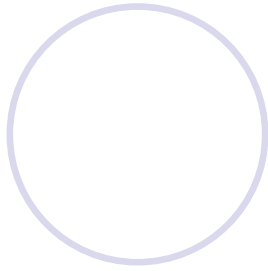


10. 以单链表作为存储结构实现直接插入排序算法。
11. 设计算法，使得在尽可能少的时间内重排数组，将所有取负值的关键字放在所有取非负值的关键字之前。请分析算法的时间复杂度。



12. 下面是一个自上往下扫描的冒泡排序的伪代码算法，它采用lastExchange 来记录每趟扫描中进行交换的最后一个元素的位置，并以它作为下一趟排序循环终止的控制值。请仿照它写一个自下往上扫描的冒泡排序算法。

```
void BubbleSort(int A[],int n)
//不妨设A[0..n-1]是整型向量
int lastExchange,j,i=n-1;
```



```
while (i>0)
```

```
    lastExchange=0;
```

```
    for(j=0;j<i;j++)//从上往下扫描A[0..i]
```

```
        if(A[j+1]<A[j]){
```

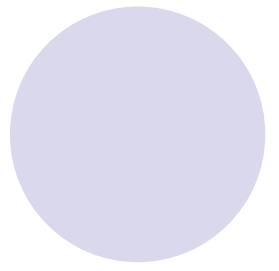
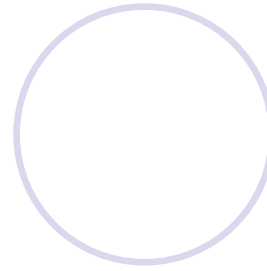
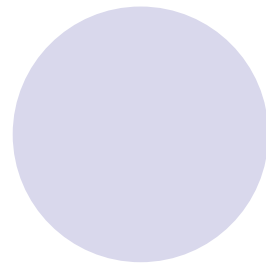
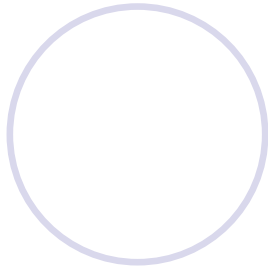
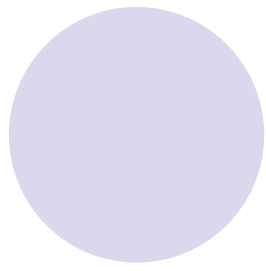
```
            交换A[j]和A[j+1];
```

```
            lastExchange=j;
```

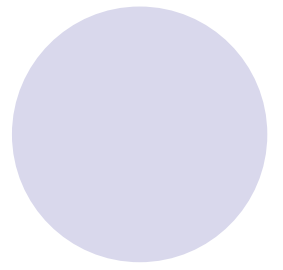
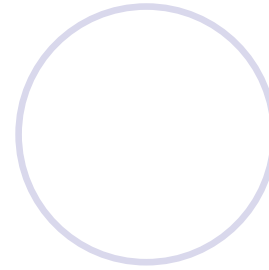
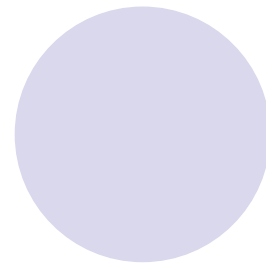
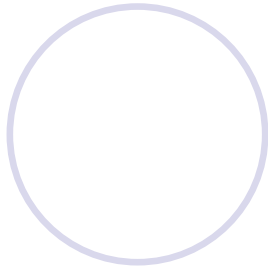
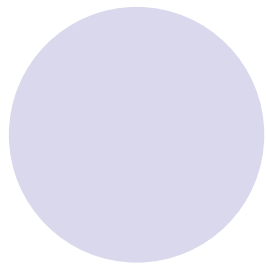
```
        }
```

```
    i=lastExchange;//将i置为最后交换的位置
```

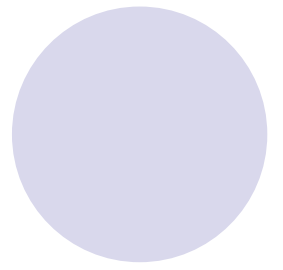
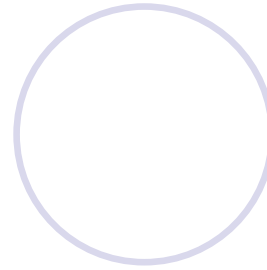
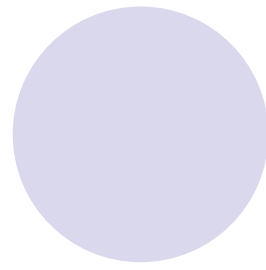
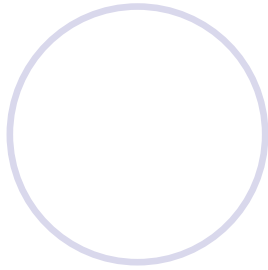
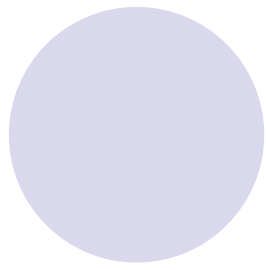
```
}//endwhile }//BubbleSort
```



13. 改写快速排序算法，要求采用三者取中的方式选择划分的基准记录；若当前被排序的区间长度小于等于3时，无须划分而是直接采用直接插入方式对其排序。
14. 对给定的 $j(1 \leq j \leq n)$ ，要求在无事的记录区 $R[1..n]$ 中找到按关键字自小到大排在第 j 个位置上的记录(即在无事的集合中找到第 j 个最小元)，试利用快速排序的划分思想编写算法实现上述的查找操作。



15. 以单链表为存储结构，写一个直接选择排序算法。
16. 写一个建堆算法：从空堆开始，依次读入元素调用上题中堆插入算法将其插入堆中。
17. 已知两个单链表中的元素递增有序，试写算法将这两个有序表归并成一个递增有序的单链表。算法应利用原有的链表结点空间。果可输出到另一个向量 $B[0..n-1]$ 中。



18. 设向量 $A[0..n-1]$ 中存有 n 个互不相同的整数，且每个元素的值均在0到 $n-1$ 之间。试写一时间为 $O(n)$ 的算法将向量 A 排序，结果可输出到另一个向量 $B[0..n-1]$ 中。